CS24301 — THEORY (3.0 CREDITS) B.TECH CSE 5TH SEMESTER BIT MESRA | NEP SCHEME

Compiler Design — Module I: Lexical Analysis

Comprehensive Study Notes, Formal Theory, Algorithms & Solved Numerical Masterclass

Table of Contents — Module I Topics

1. Language Processing Systems & Cousins of the Compiler

A **compiler** is a program that reads a source program written in a high-level programming language and translates it into an equivalent target machine language or assembly program, while diagnosing and reporting syntactic and semantic errors.

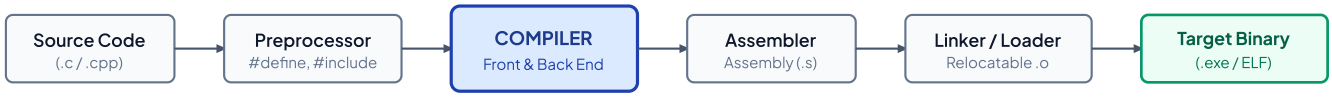


FIGURE 1.1: LANGUAGE PROCESSING TOOLCHAIN ARCHITECTURE

1.1 Detailed Functions of the Cousins of the Compiler

Tool	Input → Output	Core Functions & Responsibilities
Preprocessor	`.c / .cpp` → `.i` (Pure Source)	Expands macros (`#define`), resolves file inclusions (`#include`), strips source comments, evaluates conditional compilation flags (`#ifdef`).
Compiler	`.i` → `.s` (Assembly)	Performs full analysis (lexical, syntax, semantic) and synthesis (optimization, code gen) to produce target assembly code.
Assembler	`.s` → `.o` (Object File)	Converts assembly language mnemonics (`MOV` , `ADD`) into relocatable machine instructions and builds object symbol tables.
Linker	`.o` files + Libs → Executable	Resolves external symbol references across multiple object modules; statically links runtime libraries (e.g., `printf` in `libc.a`).
Loader	Executable → RAM Memory	Allocates primary memory segments (Code, Data, Stack, Heap), initializes registers, relocates base addresses, and jumps to entry point (`main`).

2. Structure & Phases of a Compiler

Compilation is logically organized into two super-phases: **Analysis (Front-End)** which is language-specific and machine-independent, and **Synthesis (Back-End)** which is target-architecture dependent.

1. Lexical Analysis (Scanner)
Scans character stream from left-to-right, groups them into *lexemes*, and outputs a stream of *tokens* $\langle \text{token\_name}, \text{attr\_val} \rangle$. Strips comments and blanks.

2. Syntax Analysis (Parser)
Imposes hierarchical grammatical structure using a Context-Free Grammar (CFG). Constructs a *Parse Tree* or *Abstract Syntax Tree (AST)*.

3. Semantic Analysis (Type Checker)
Verifies language semantic consistency. Performs *Type Checking* and inserts necessary *Type Coercions* (e.g., `inttofloat`).

4. Intermediate Code Generator (ICG)
Generates an explicit, machine-independent intermediate form, typically *Three-Address Code (TAC)* where instructions have at most 1 operator on RHS.

5. Code Optimizer
Transforms intermediate code for faster execution and smaller memory footprint (Constant Folding, Common Subexpression Elimination, Dead Code Removal).

6. Target Code Generator
Translates optimized intermediate representation into target machine assembly/binary code, performing *instruction selection* and *register allocation*.

2.1 Running Example: Translation Trace of `position = initial + rate * 60`

Compilation Phase	Transformed Code / Representation
Source Statement	`position = initial + rate * 60`
1. Lexical Analysis	`<id, 1> ⟺ <id, 2> <+> <id, 3> <*> <int_const, 60>` (Symbol Table: 1→position, 2→initial, 3→rate)
2. Syntax Analysis	Abstract Syntax Tree with root `=`, left child `<id,1>`, right child `+(<id,2>, *(<id,3>, 60))`
3. Semantic Analysis	Decorated AST: inserts `inttofloat(60)` node because `rate` is declared as `float`.

4. Intermediate Code (3AC)	`t1 = inttofloat(60)`
	`t2 = id3 * t1`
	`t3 = id2 + t2`
	`id1 = t3`

5. Code Optimization	`t1 = id3 * 60.0` *(Constant folded at compile-time; temporary t3 eliminated)* `id1 = id2 + t1`

6. Target Code Gen	`LDF R2, id3`
	`MULF R2, R2, #60.0`
	`LDF R1, id2`
	`ADDF R1, R1, R2`
	`STF id1, R1`

3. Role and Mechanics of the Lexical Analyzer

3.1 Fundamental Distinction: Token, Pattern, and Lexeme

Concept	Formal Definition	Representation Form	Concrete Examples
Token	Abstract syntactic terminal symbol consumed by the parser.	Grammar Terminal (e.g., `ID`, `IF`, `NUM`, `RELOP`)	`<ID, ptr>`, `<IF>`, `<RELOP, LE>`
Pattern	The formal specification/rule describing the set of matching strings.	Regular Expression (RE)	`[a-zA-Z_][a-zA-Z0-9_]*`
Lexeme	The concrete character sequence from the source code matching the pattern.	Raw string of characters	`count`, `total_score`, `3.1415`

3.2 Lexical Errors & Recovery Techniques

When the scanner encounters a character stream that matches no defined token pattern, it applies recovery strategies:

- **Panic Mode:** Successively discards incoming characters until a recognized delimiter (e.g., whitespace, semicolon) is reached.
- **Character Deletion:** Removes an invalid character (e.g., `x = 10 $ 5;` $\rightarrow$ remove `$`).
- **Character Insertion:** Inserts a missing delimiter (e.g., inserting a closing quotation mark).
- **Character Substitution:** Replaces an invalid symbol with a valid candidate.
- **Character Transposition:** Swaps two adjacent transposed characters (e.g., correcting `hte` to `the`).

4. Input Buffering & Sentinel Optimization

Reading characters one-by-one with OS system calls (e.g., `getc()`) causes severe performance bottlenecks. Compilers employ **Buffer Pairs** (size $N = 4096$ bytes) with **Sentinel (EOF) Characters**:

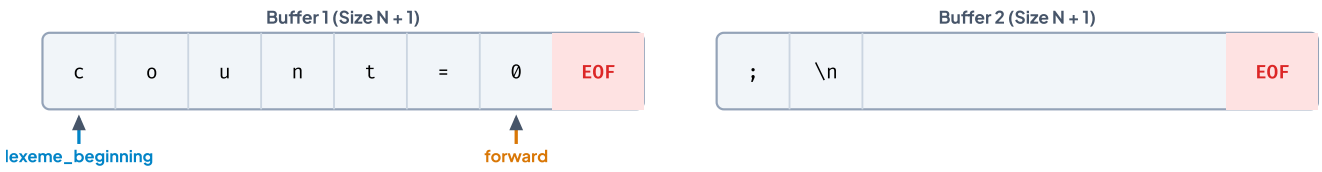


FIGURE 4.1: BUFFER PAIRS WITH SENTINEL EOF CHARACTERS

Sentinel Comparison Reduction Principle

Standard Scanner Loop: Requires **2 tests** per character: (1) Check if `forward == buffer_end`, and (2) test character value.
Sentinel-Optimized Loop: Requires only **1 test** per character: `switch (*forward++)`. The special `EOF` case handles buffer wrapping only when the buffer end is actually reached.

5. Regular Expressions & Finite Automata (NFA vs. DFA)

5.1 Algebraic Laws of Regular Expressions

Law Name	Algebraic Identity Formulation
Commutativity of Union	$r \mid s = s \mid r$
Associativity	$(r \mid s) \mid t = r \mid (s \mid t) \quad \text{and} \quad (rs)t = r(st)$
Distributivity	$r(s \mid t) = rs \mid rt \quad \text{and} \quad (s \mid t)r = sr \mid tr$
Identity Elements	$r \mid \emptyset = r \quad \text{and} \quad r\epsilon = \epsilon r = r$
Kleene Closure Laws	$r^* = (r \mid \epsilon)^*, \quad (r^*)^* = r^*, \quad r^* = \epsilon \mid rr^*$

5.2 NFA vs. DFA Comparison

Feature	Non-Deterministic Finite Automata (NFA)	Deterministic Finite Automata (DFA)
ϵ-Transitions	Allowed (ϵ-moves without reading input).	Strictly prohibited.
Next State Determinism	Set of multiple potential next states.	Exactly one unique next state for each symbol.
Transition Function δ	$\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$	$\delta : Q \times \Sigma \to Q$
Time Complexity	$O(\lvert V\rvert \cdot \lvert E\rvert)$ per string (tracking state sets).	$O(\lvert s\rvert)$ where each step is an $O(1)$ table lookup.

6. Thompson's Construction Algorithm (RE $\to$ NFA)

Constructs an equivalent NFA $N(r)$ by structural induction with the invariant of exactly **one start state** and **one accepting state**:

FIGURE 6.1: THOMPSON'S CONSTRUCTION STRUCTURAL TEMPLATES

7. Direct DFA Construction from Regular Expressions (Syntax Tree Method)

Direct construction avoids intermediate NFA and subset construction overhead by directly computing four functions on the **Syntax Tree of the Augmented Regular Expression** $(r)\#$:

Function	Return Type	Formal Recursive Rules & Intuitive Meaning
`nullable(n)`	Boolean	• Leaf ϵ: `true` | Leaf i (symbol or $\#$): `false` • Or-node $c_1 \mid c_2$: $\text{nullable}(c_1) \vee \text{nullable}(c_2)$ • Cat-node $c_1 \cdot c_2$: $\text{nullable}(c_1) \wedge \text{nullable}(c_2)$ • Star-node c_1^*: `true`
`firstpos(n)`	Set of Positions	• Leaf ϵ: $\emptyset$ | Leaf i: $\{i\}$ • Or-node $c_1 \mid c_2$: $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ • Cat-node $c_1 \cdot c_2$: $\begin{cases} \text{firstpos}(c_1) \cup \text{firstpos}(c_2) & \text{if nullable}(c_1) \\ \text{firstpos}(c_1) & \text{otherwise} \end{cases}$ • Star-node c_1^*: $\text{firstpos}(c_1)$
`lastpos(n)`	Set of Positions	• Leaf ϵ: $\emptyset$ | Leaf i: $\{i\}$ • Or-node $c_1 \mid c_2$: $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ • Cat-node $c_1 \cdot c_2$: $\begin{cases} \text{lastpos}(c_1) \cup \text{lastpos}(c_2) & \text{if nullable}(c_2) \\ \text{lastpos}(c_2) & \text{otherwise} \end{cases}$ • Star-node c_1^*: $\text{lastpos}(c_1)$
`followpos(i)`	Set of Positions	**Rule 1 (Cat-node $c_1 \cdot c_2$):** For every $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_2)$. **Rule 2 (Star-node c_1^*):** For every $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_1)$.

8. Solved Master Numerical Walkthrough: $(a|b)^*abb\#$

Step 1: Leaf Position Indexing & AST Evaluation Table

Leaves numbered: $a(1), b(2), a(3), b(4), b(5), \#(6)$.

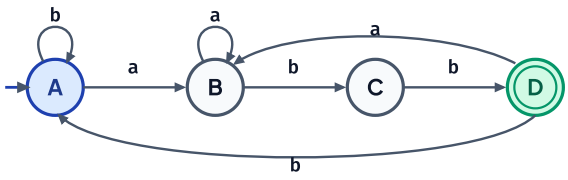
Node	Subtree Expression	`nullable`	`firstpos`	`lastpos`
1	Leaf a	`false`	$\{1\}$	$\{1\}$
2	Leaf b	`false`	$\{2\}$	$\{2\}$
or_1	$a \mid b$	`false`	$\{1,2\}$	$\{1,2\}$
star_1	$(a \mid b)^*$	**`true`**	$\{1,2\}$	$\{1,2\}$
3	Leaf a	`false`	$\{3\}$	$\{3\}$
cat_2	$(a \mid b)^*a$	`false`	$\{1,2,3\}$	$\{3\}$
4	Leaf b	`false`	$\{4\}$	$\{4\}$
cat_3	$(a \mid b)^*ab$	`false`	$\{1,2,3\}$	$\{4\}$
5	Leaf b	`false`	$\{5\}$	$\{5\}$
cat_4	$(a \mid b)^*abb$	`false`	$\{1,2,3\}$	$\{5\}$
6	Leaf $\#$	`false`	$\{6\}$	$\{6\}$
cat_5 **(Root)**	$(a \mid b)^*abb\#$	`false`	$\{1,2,3\}$	$\{6\}$

Step 2: Computed `followpos` Values

Position i	Symbol	$\text{followpos}(i)$ Set Calculation Rationale
1	a	$\{1, 2, 3\}$ (From star_1 rule $\{1, 2\}$ and cat_2 rule $\{3\}$)
2	b	$\{1, 2, 3\}$ (From star_1 rule $\{1, 2\}$ and cat_2 rule $\{3\}$)
3	a	$\{4\}$ (From cat_3 rule: $\text{firstpos}(4) = \{4\}$)
4	b	$\{5\}$ (From cat_4 rule: $\text{firstpos}(5) = \{5\}$)
5	b	$\{6\}$ (From cat_5 rule: $\text{firstpos}(6) = \{6\}$)
6	$\#$	$\emptyset$ (Endmarker has no succeeding positions)

Step 3: DFA State Transitions & State Diagram

DFA State	NFA Position Set	On a	On b	Accepting?
$\rightarrow \mathbf{A}$	$\{1, 2, 3\}$	$\mathbf{B}$	$\mathbf{A}$	No
$\mathbf{B}$	$\{1, 2, 3, 4\}$	$\mathbf{B}$	$\mathbf{C}$	No
$\mathbf{C}$	$\{1, 2, 3, 5\}$	$\mathbf{B}$	$\mathbf{D}$	No
$*\mathbf{D}$	$\{1, 2, 3, 6\}$	$\mathbf{B}$	$\mathbf{A}$	**YES** (contains 6)



FINAL MINIMAL 4-STATE DFA

9. High-Yield Exam Question Bank & Solved Numericals

Q1. Why are Lexical Analysis and Syntax Analysis separated into two distinct phases?

Answer:

1. **Simpler Design:** Allows grammar rules (CFGs) to focus entirely on hierarchical structure without being polluted by whitespace and comment rules.
2. **Compiler Efficiency:** Specialized lexical tokenizers built as DFAs process input in $O(1)$ time per character with optimized buffering.
3. **Portability:** Device/platform character set idiosyncrasies are isolated entirely inside the lexer.

Q2. Given the DFA: States $\{q_0, q_1, q_2, q_3, q_4\}$, Start q_0, Accept $\{q_4\}$. Transitions: $\delta(q_0, a) = q_1, \delta(q_0, b) = q_2$; $\delta(q_1, a) = q_1, \delta(q_1, b) = q_3$; $\delta(q_2, a) = q_1, \delta(q_2, b) = q_2$; $\delta(q_3, a) = q_1, \delta(q_3, b) = q_4$; $\delta(q_4, a) = q_1, \delta(q_4, b) = q_2$. Perform State Minimization.

Solution:

1. **Initial Partition P_0:** Group $G_1 = \{q_0, q_1, q_2, q_3\}$ (Non-accepting), Group $G_2 = \{q_4\}$ (Accepting).
2. **Refine $P_0 \to P_1$:** On input b, $\delta(q_3, b) = q_4 \in G_2$, whereas q_0, q_1, q_2 transition to states within G_1. Thus q_3 splits: $P_1 = \{\{q_0, q_1, q_2\}, \{q_3\}, \{q_4\}\}$.
3. **Refine $P_1 \to P_2$:** On input b, $\delta(q_1, b) = q_3 \in \{q_3\}$, whereas $\delta(q_0, b) = q_2$ and $\delta(q_2, b) = q_2 \in \{q_0, q_1, q_2\}$. Thus q_1 splits: $P_2 = \{\{q_0, q_2\}, \{q_1\}, \{q_3\}, \{q_4\}\}$.
4. **Check $P_2 \to P_3$:** Testing $\{q_0, q_2\}$ on $a \to q_1 \in \{q_1\}$, on $b \to q_2 \in \{q_0, q_2\}$. No further splits occur.
5. **Final Minimized DFA:** 4 distinct states: $[q_0, q_2]$ (Start), $[q_1]$, $[q_3]$, and $[q_4]$ (Final Accepting).

10. Quick Revision Cheat Sheet & Top Exam Traps

⚠ **Top 5 High-Probability Exam Traps:**

- **Always append endmarker $\#$:** In Direct DFA construction, failing to augment $(r)\#$ means accepting states cannot be identified.
- **DFA Acceptance Condition:** Any DFA state subset containing the position of $\#$ is an accepting state.
- **Sentinel Placement:** The sentinel `EOF` is placed at the end of *both* buffers (Buffer 1 and Buffer 2), reducing per-character comparisons from 2 to 1.
- **Cat-Node Followpos:** For $c_1 \cdot c_2$, $\text{firstpos}(c_2)$ is added to $\text{followpos}(i)$ for *all* $i \in \text{lastpos}(c_1)$.
- **Token vs Lexeme:** A token is the generic class name (e.g., `ID`), whereas a lexeme is the literal concrete string (e.g., `sum`).